

Interface - Deep drive

- Interfaces are the blue print of behaviour.
- Any class that implements INTERFACE should necessarily define the each behaviors.
- Therefore INTERFACE is also known as CONTRACT.
- Every method in interface is by-default public.

Dynamic Polymorphism ka bhi ache se use hota ha, its code is given below.

```
public class DynamicPolymorphism {
    public static void main(String[] args) {
        Payment p = new DebitCard();// dynamic action yha hoga, jo bhi class dalegi uska hi method call hoga.
        p.pay();
    }
}
// Dynamic polymorphism or Dynamic Dispatch.// it means ki ye run time pr decide krta ha.
interface Payment{
    void pay();
}
class CreditCard implements Payment{
    @Override
    public void pay(){
        System.out.println("Paying via Credit Card");
    }
}
class DebitCard implements Payment{
    @Override
    public void pay(){
        System.out.println("Paying via Debit Card");
    }
}
```

Variables inside interface

- Vo variables defaultly “ public static final” hote ha.

```
public class VariablesInInterfaces {
    public static void main(String[] args) {
        MathConstant r1 = new Random();
        r1.fun();
        System.out.println(MathConstant.VALUE);
    }
}
// Variables inside Interfaces-->
interface MathConstant {
    double PI_VALUE = 3.14;// ye ek constant ha to interface me apne aap "public static final" internally lg jata ha.
    int VALUE = 10;
    void fun();
}
class Random implements MathConstant{
    public void fun(){
        System.out.println(PI_VALUE +", "+ VALUE);
    }
}
```

Multiple inheritance interface

```
// Multiple inheritance can be possible through interfaces--> Parent-A and Parent-B both are extended by child.
interface A {
    void fun();
}
interface B {
    void fun2();
}
class C implements A, B{
```

```

@Override
    public void fun(){
    }

@Override
    public void fun2(){
    }
}

```

Interface Inheritance

```

// interface inheritance.
interface Animal{
    void eat();
}

interface Dog extends Animal{//when interface inheritance another interface the keyword used is "extends".
    void bark();
}

class StreetDog implements Dog{
    @Override
    public void eat(){
        System.out.println("eating");
    }

    @Override
    public void bark(){
        System.out.println("barking");
    }
}

```

“Default Method”, “static Method”, “private access modifier.

- Each and every method in interface are by default “ public abstract void drive(); ”
- But after java 8, we can also declare the method in interface.
- “Default” keyword is added to declare the method in interface.

```

interface Vehicle{
    // "Default Method".
    default void drive(){// default keyword is used to define the method.
        System.out.println("Vehicle is driving");
        accelerate(); // private will call through here.
    }

    // "Static Method".
    static void brake(){
        System.out.println("vehicle is applying brake");
    }

    // "private Method".
    private void accelerate(){// because this is private, so interface k bhar se call ni kr skte.
        System.out.println("Vehicle is Accelerating");
    }
}

class Car implements Vehicle{
    // i can also @override the drive(); method, if i want.
}

```

Diamond problem fix by interface: MULTIPLE INHERITANCE

```

public class DiamondProblemInterface {
    public static void main(String[] args) {

```

```
D d = new D();
d.fun();
}
}
interface A {
    void fun();
}
interface B extends A {
    default void fun(){
        System.out.println("B interface.");
    }
}
interface C extends A {
    default void fun(){
        System.out.println("C interface.");
    }
}
class D implements B, C {
    @Override // kuki B, C dono interfaces ke method defined ha to yha @override krna hi pdega.
    public void fun(){
        System.out.println("class D.");
        // now if i want to call fun method of B, C then-->
        B.super.fun();
        C.super.fun();
    }
}
```

Resolution priority rule:

```
public class ResolutionPriorityRule {
    public static void main(String[] args) {
        C c = new C();
        c.fun();
    }
}
/*
java resolution priority rule-->agar class c exteds class B and also implements interface A ,
and both "class B & interface A " has same method,
so, priority will be given to the class method always by default.
*/
interface A {
    default void fun(){
        System.out.println("Inside interface A");
    }
}
class B {
    public void fun(){
        System.out.println("inside class B");
    }
}
class C extends B implements A {
    //agar me chahta hu ki interface ka method call ho to-->
    @Override
    public void fun(){
        A.super.fun();
    }
}
```

<u>Interfaces</u>	<u>Abstract Classes</u>
No variables are allowed, because of public static	Variables are allowed.

and final.	
No Constructors are allowed.	Constructors are allowed.
Multiple inheritance is possible	Multiple inheritance is not possible.
Methods in inheritance are generally public.	Methods can be of any access modifier.

Functional interfaces: These interfaces unlocks important concept of “functional programming”, using **LAMDA** expressions.

- Ye wo interfaces hote ha jinme sirf ek hi functional method ho, or bs wo declare ho.
- Example: **interface A {
 Void fun(); }**
- **Some examples are**→ comparable interface, predicate interface, etc.

Marker interfaces: these interfaces are totally empty. With no methods inside it.

- Example: **interface A { }.**

	Types of Marker interfaces →
1. Interface Clonable.	
2. interface Serializable.	
3. Interface RandomAccess.	

Note:

- Interface Animal →internaly when compile java code then it will convert to “class Animal” or isko java is tarah se baki class se alag krega ki usme ACC_Interface ka tag lga dega, jo ye btaega ki ye koi class nahi blki interface ha.